Clone and run the T14 project

A practical handoff for a new contributor

Project: Yamin310702/T14-Affordance-Project

Prepared: October 27, 2026. Commands use Bash/Zsh on macOS or Linux. Windows users should use an Ubuntu WSL2 terminal and follow the Linux/CPU route.

What you can run

Stage	Purpose
Baseline diagnostics	Evaluate frozen OpenAD PN2 on partial validation views.
E17	Compare training-associated names with replacement query names.
E18	Test fixed prompt transfer on five actions, including negative-object alarms.
Historical Espresso work	Review archived results. Re-running needs a separate unofficial model stack.

This repository is a research handoff. Git clone provides scripts, records and the current proposal. You must separately obtain the complete OpenAD source, data and model checkpoint. There is no single command that reproduces every historical experiment.

VideoAfford is the selected conference paper. This guide does not implement or train VideoAfford. The proposed 18-action, three-prompt OpenAD ensemble is also a future experiment, not a completed runnable pipeline.

Recommended route

Follow pages 2-6 in order. Run all three smoke checks before the larger evaluations on pages 7-9. Review troubleshooting and reproduction limits on pages 10-11.

Prerequisites: Git, Conda or Miniforge, an internet connection for downloads, and sufficient free disk and memory. For this CPU/MPS path, reserve about 15 GB of free disk as a planning allowance, not a measured minimum. CPU execution can be slow.

A clean-machine installation was not performed for this guide. The archived OpenAD patch and path-edit logic were checked against local source copies. Package availability, downloaded assets and device support must still pass the checks below.

1. Clone both repositories

Install Git and Conda first if needed. Use the official Conda installation instructions [4]. Open a new terminal and verify:

```
git --version
conda --version
```

Choose a new working directory. If these repository folders already exist, use another directory instead of cloning over them. Run the following in the same terminal:

```
mkdir -p "$HOME/T14-work"
cd "$HOME/T14-work"
export T14_WORK="$PWD"

git clone https://github.com/Yamin310702/T14-Affordance-Project.git
export T14_PROJECT="$T14_WORK/T14-Affordance-Project"

export OPENAD_URL="https://github.com/Fsoft-AIC/"
export OPENAD_NAME="Open-Vocabulary-Affordance-Detection-in-3D-Point-Clouds"
git clone "${OPENAD_URL}${OPENAD_NAME}.git" "$T14_WORK/OpenAD"
export OPENAD_REPO="$T14_WORK/OpenAD"
```

Use the archived OpenAD source revision

The project records this upstream revision. Detaching HEAD is expected here. The next page applies the archived local adaptations to this source.

```
git -C "$OPENAD_REPO" checkout \
  b082265ed085455a3fdc80b0d8d95dc51351e7b4

git -C "$T14_PROJECT" rev-parse HEAD
git -C "$OPENAD_REPO" rev-parse HEAD
```

Save both printed revision IDs with each new experiment. The project repository is public, so cloning it does not require collaborator permission. Pushing changes still requires write access.

Expected layout

```
T14-work/
  T14-Affordance-Project/    # team scripts and evidence
  OpenAD/                  # external baseline source and assets
```

2. Create a separate Python environment

Create a new environment so existing research environments remain intact. Python 3.10 matches the major/minor version used by the archived Mac work.

```
conda create -n t14-openad python=3.10 -y
conda activate t14-openad
python --version
python -m pip install --upgrade pip
```

For this CPU/MPS route, install PyTorch and torchvision together. Use the official selector [3] if your OS or Python build needs a different command. Native CUDA acceleration is outside this guide because E17/E18 currently accept only cpu or mps.

```
python -m pip install torch torchvision
python -m pip install numpy tqdm h5py transforms3d scikit-learn
python -m pip install gorilla-core==0.2.7.8 tensorboardX
python -m pip install matplotlib ftfy regex packaging
python -m pip install git+https://github.com/openai/CLIP.git
python -m pip check
```

Install **scikit-learn**, not the deprecated **sklearn** package name. tensorboardX is required by OpenAD imports even for the validation-only diagnostic script. The separate PyPI package named clip is not a substitute for OpenAI CLIP [5].

Verify the imports

```
python - <<'PY'
import sys
import torch, torchvision, numpy, h5py, sklearn
import clip, tensorboardX, transforms3d
from gorilla.config import Config
print("Python:", sys.version)
print("PyTorch:", torch.__version__)
print("MPS available:", torch.backends.mps.is_available())
print("Imports OK")
PY
```

Expected: **Imports OK**. Do not continue past a missing-package error. These are setup instructions, not a fully pinned reproduction environment. Save the actual installed versions on page 7.

3. Apply adaptations and make paths portable

The original OpenAD uses CUDA-specific calls. Apply the team's archived changes to the fresh, pinned OpenAD clone. This step is for that clone only; do not reapply it to an already modified checkout.

```
export T14_PATCH="$T14_PROJECT/research_archive/2026-10-05/code/"
export T14_PATCH="${T14_PATCH}openad/working_changes.patch"
git -C "$OPENAD_REPO" apply --check "$T14_PATCH"
git -C "$OPENAD_REPO" apply "$T14_PATCH"
git -C "$OPENAD_REPO" status --short
```

Expected modified files: models/openad_pn2.py, models/openad_dgcnn.py, test_open_vocab.py and utils/eval.py. If the check fails, stop and confirm the revision and whether the patch was already applied.

Replace the original author's path in E17/E18

This one-time edit makes the two active runners read OPENAD_REPO from your terminal. It changes your local project clone only. Keep dated archive scripts and run records unchanged.

```
python - <<'PY'
import os
from pathlib import Path
root = Path(os.environ["T14_PROJECT"])
old = ('REPO = Path("/Users/yaminmyataung/'
       'Open-Vocabulary-Affordance-Detection-in-3D-Point-Clouds")')
new = 'REPO = Path(os.environ["OPENAD_REPO"]).expanduser().resolve()'
for name in ["run_seen_query_comparison.py",
             "run_e18_openad_prompt_transfer.py"]:
    path = root / "pilot" / name
    text = path.read_text()
    if new in text:
        print(name, "already configured")
        continue
    assert text.count(old) == 1, (name, "unexpected source")
    path.write_text(text.replace(old, new, 1))
    print(name, "configured")
PY
```

analyze_dataset_scores.py already accepts --repo, so it needs no source edit. None of these setup changes trains a model or changes its checkpoint.

4. Download data and pretrained weights

Open the official OpenAD README [2] and use its **Data** and **pretrained models** links. If a Drive link requires permission, resolve that access before continuing. Downloads are manual in this guide.

Data folder: open Google Drive
Pretrained checkpoint folder: open Google Drive

```
mkdir -p "$OPENAD_REPO/data" "$OPENAD_REPO/checkpoints"
```

Extract the downloaded archives. Place the files directly in these directories, with exactly these names:

```
OpenAD/
  data/
    partial_view_val_data.pkl
    full_shape_val_data.pkl
  checkpoints/
    best_model_openad_pn2_partial.t7
```

Required first: partial_view_val_data.pkl and the partial checkpoint. The full validation file is needed only for the E17 full-geometry comparison. The chosen validation-only runners do not need training data or class-weight arrays. Do not download or load pickle/checkpoint files from untrusted sources.

Check files and checkpoint identity

```
python - <<'PY'
import os, hashlib
from pathlib import Path
r = Path(os.environ["OPENAD_REPO"])
for rel in ["data/partial_view_val_data.pkl",
            "checkpoints/best_model_openad_pn2_partial.t7"]:
    p = r / rel
    assert p.is_file(), f"Missing: {p}"
    print(rel, p.stat().st_size, "bytes")
p = r / "checkpoints/best_model_openad_pn2_partial.t7"
print("SHA256:", hashlib.sha256(p.read_bytes()).hexdigest())
PY
```

Archived partial-checkpoint SHA-256:

```
cbcfe9ff70448d97a7047f1ade817645a9be29dd2782fba57776792a8210f2ae
```

A different hash means a different asset. Check the download and filename before treating a run as a reproduction.

5. Select the device and run smoke checks

Keep CUDA hidden for this CPU/MPS route so OpenAD's automatic text-encoder placement matches the supported E17/E18 device choices. On an Apple Silicon Mac use MPS when available; otherwise use CPU.

```
export CUDA_VISIBLE_DEVICES=""
export OMP_NUM_THREADS=4
export MKL_NUM_THREADS=4
export T14_DEVICE="$(python -c 'import torch; print(
    "mps" if torch.backends.mps.is_available() else "cpu")')"
echo "$T14_DEVICE"

cd "$T14_PROJECT"
mkdir -p local_runs
```

The first model import may download OpenAI CLIP ViT-B/32 weights. Keep the internet connection available. Run the following commands one at a time.

A. Baseline: eight partial views

```
python analyze_dataset_scores.py \
  --repo "$OPENAD_REPO" --limit 8 \
  --output "$T14_PROJECT/local_runs/baseline_smoke"
```

B. E17: paired query comparison on eight views

```
python pilot/run_seen_query_comparison.py \
  --split partial --limit 8 --batch-size 4 --threads 4 \
  --device "$T14_DEVICE" \
  --output-dir "$T14_PROJECT/local_runs/e17_smoke"
```

C. E18: fixed prompt transfer on two eligible objects

```
python pilot/run_e18_openad_prompt_transfer.py \
  --max-shapes 2 --device "$T14_DEVICE" \
  --output-dir "$T14_PROJECT/local_runs/e18_smoke"
```

These are execution checks, not meaningful performance estimates. In a tiny E18 subset some positive/negative pools can be empty, so null summaries are expected. Do not infer a prompt improvement from two objects.

6. Check success and run the complete baseline

All three metadata files must report complete before you start longer evaluations:

```
python - <<'PY'
import json, os
from pathlib import Path
r = Path(os.environ["T14_PROJECT"]) / "local_runs"
checks = [r / "baseline_smoke/summary.json",
          r / "e17_smoke/partial_validation/run.json",
          r / "e18_smoke/run.json"]
for p in checks:
    info = json.loads(p.read_text())
    print(p, info["status"])
    assert info["status"] == "complete"
PY
```

Save your actual environment

```
python -m pip freeze > "$T14_PROJECT/local_runs/environment.txt"
conda env export --no-builds \
  > "$T14_PROJECT/local_runs/conda_environment.yml"
git -C "$T14_PROJECT" rev-parse HEAD \
  > "$T14_PROJECT/local_runs/project_commit.txt"
git -C "$OPENAD_REPO" rev-parse HEAD \
  > "$T14_PROJECT/local_runs/openad_commit.txt"
git -C "$OPENAD_REPO" diff \
  > "$T14_PROJECT/local_runs/openad_local_changes.patch"
git -C "$T14_PROJECT" diff -- pilot \
  > "$T14_PROJECT/local_runs/runner_path_changes.patch"
```

Complete partial-view baseline evaluation

```
python analyze_dataset_scores.py \
  --repo "$OPENAD_REPO" \
  --output "$T14_PROJECT/local_runs/baseline_full"
```

Omitting --limit evaluates the entire loaded validation split. The archived run had 9,140 views. Compare against your own reported dataset count rather than assuming every download is identical.

Inspect baseline_full/summary.json for status, completed_samples, device, checkpoint_sha256 and miou_official_epsilon. IoU is stored as a fraction: 0.125 means 12.5%. This baseline averages 19 classes including none, not only 18 actions.

The run also writes per_affordance.csv, confusion_counts.csv, sample_regions.csv and aggregates.npz. It does not compute negative-object alarm rates. Use E18 for that analysis.

7. Run E17: query-vocabulary comparison

E17 extracts point features once per input and scores both aligned query lists. Its output separates the 18-action mean IoU from the 19-class mean IoU including none.

Complete partial-view comparison

```
python pilot/run_seen_query_comparison.py \
  --split partial --batch-size 4 --threads 4 \
  --device "$T14_DEVICE" \
  --output-dir "$T14_PROJECT/local_runs/e17_full"
```

Optional: full-geometry control

Only run this after placing full_shape_val_data.pkl in OpenAD/data. The current E17 script reuses the partial-view checkpoint for both geometry settings. This is not the separate historical E01 full-shape checkpoint baseline.

```
python pilot/run_seen_query_comparison.py \
  --split full --batch-size 4 --threads 4 \
  --device "$T14_DEVICE" \
  --output-dir "$T14_PROJECT/local_runs/e17_full"
```

File under each split directory	What it contains
run.json	Device, checkpoint hash, completion status and aggregate metrics for both vocabularies.
per_class_iou.csv	Intersections, unions and IoU for every query slot.
per_view_metrics.csv	Per-view records, object IDs and condition labels.

The split directories are partial_validation/ and full_validation/. Compare summary.training_name_queries and summary.heldout_query_names inside run.json.

Interpretation: many paired names are synonyms. A score gap establishes vocabulary sensitivity under this protocol. It does not establish generalization to entirely new action concepts. Do not use a smoke subset as the final reported result.

Use a fresh destination if repeating a run. These scripts overwrite existing output files instead of resuming an interrupted job.

8. Run E18: localization and false alarms

E18 tests five fixed actions under canonical-word, original-sentence and selected-paraphrase conditions. It excludes object IDs from earlier prompt studies by reading the archived E09-E13 metadata. Those archive folders must remain in your project clone.

```
python pilot/run_e18_openad_prompt_transfer.py \
  --device "$T14_DEVICE" \
  --output-dir "$T14_PROJECT/local_runs/e18_full"
```

File	Use
run.json	Completion, prompts, checkpoint, dataset counts and paired uncertainty summaries.
per_affordance_summary.csv	Global IoU, positive-object IoU/AUC and negative-object alarm summaries.
per_object_metrics.csv	Paired condition records for each object and action.
object_manifest.csv	Objects, counts and condition-specific diagnostic values.
selected_test_shape_ids.txt	Exact selected IDs for this run.
excluded_prior_shape_ids.txt	Earlier study IDs excluded by the script.

Read the metrics together

Global IoU: overlap of target predictions and labels after the fixed 19-way argmax. **Negative-object false-positive fraction:** fraction of points assigned the target on an object whose labels contain no target. **Any-alarm rate:** fraction of those negative objects with at least one target prediction.

A prompt that increases IoU but also increases alarms has a trade-off. E18 is exploratory validation, not an untouched confirmatory test. Its target actions and comparisons differ from the proposed 18-action, three-paraphrase score-map study.

End-of-run checks

Require status = complete. Record completed_shapes, selected_shape_count, evaluated_views, device and checkpoint_sha256. Read the denominator for each action before interpreting alarm rates or confidence intervals.

Keep the canonical baseline, prompts and object selection fixed. Do not pick new prompts after inspecting the final reported validation results.

9. Troubleshooting and repeat sessions

Symptom	Action
conda: command not found	Install Conda/Miniforge, initialize the shell as instructed by the installer, and reopen Terminal.
No module named gorilla / tensorboardX / clip	Activate t14-openad. Revisit page 3. Install OpenAI CLIP from its GitHub URL.
Missing /Users/yaminmyataung/...	The local runner edit was skipped. Apply page 4 to pilot/ scripts, not archive copies.
OPENAD_REPO or T14_PROJECT KeyError	Restore the environment variables using the commands below.
Text encoder and --device disagree	Hide CUDA before Python starts and recalculate T14_DEVICE. On Mac it should match MPS availability.
MPS was requested but is unavailable	Use CPU on an unsupported machine. Keep CPU and MPS results separately labeled.
Out of memory	Stop overlapping jobs. For E17 try --batch-size 1, documenting the change. CPU does not eliminate RAM requirements.
Patch does not apply	Use the pinned upstream revision and a fresh clone. Do not force or repeatedly apply the patch.
Checkpoint state-dict mismatch	Verify the PN2 partial checkpoint filename, SHA-256 and upstream source revision.
CLIP download or certificate error	Check access to the official download endpoint, proxy and certificates. Do not disable TLS validation.

Whenever you open a new terminal

```
conda activate t14-openad
export T14_WORK="$HOME/T14-work"
export T14_PROJECT="$T14_WORK/T14-Affordance-Project"
export OPENAD_REPO="$T14_WORK/OpenAD"
export CUDA_VISIBLE_DEVICES=""
export OMP_NUM_THREADS=4
export MKL_NUM_THREADS=4
export T14_DEVICE="$(python -c 'import torch; print(
    "mps" if torch.backends.mps.is_available() else "cpu")')"
cd "$T14_PROJECT"
```

10. Scope, collaboration and references

What the remaining project files mean

The dated research_archive directories preserve earlier evidence. Review their README.md, RESULTS.md and run.json files. They are not a list of independent commands that can all run in arbitrary order.

E08-E16 depend on an unofficial Espresso checkout, its own dependencies and downloaded weights/cache, plus previous-run manifests and outputs. Several scripts contain original local paths, offline-only cache settings and fixed output destinations. Reproducing them needs a separate setup audit. This guide does not claim that running those scripts after OpenAD setup will work.

Affostruction inference was not completed locally. VideoAfford is a paper-presentation topic. The proposed OpenAD prompt-ensemble experiment remains unimplemented. No new training pipeline is supplied here.

Share a new run without overwriting history

```
git switch -c yourname/reproduction-notes
git status --short
```

Review exactly which files you intend to share. Commit selected small summaries and environment records, then push the branch and open a pull request if you have write permission. Keep datasets, model checkpoints and large generated outputs out of Git. New local_runs/ files are not automatically ignored by the current repository, so avoid git add . for experiment uploads.

Do not commit full conda environment exports or logs until you review them for machine-specific paths and other information you do not intend to publish. Preserve exact run settings and method limitations in the shared summary.

Sources and verification

[1] Team code inspected at project commit 070ef74: analyze_dataset_scores.py; pilot/run_seen_query_comparison.py; pilot/run_e18_openad_prompt_transfer.py; MPS_RERUN_GUIDE.md; dated archive metadata.

[2] Official OpenAD source and asset links

[3] PyTorch installation selector

[4] Conda installation guide

[5] OpenAI CLIP source and installation

Verified during preparation: the archived patch applies to its recorded upstream revision; the portable runner substitutions compile and read an external checkout path; PDF command syntax and page layouts were checked. Not verified: a fresh dependency installation, access to every Drive asset, complete inference reruns or equality to historical metrics.